

Rapport de Projet

TP Final CI/CD — TaskFlow API

Rémi PORQUET

Juin 2026

`github.com/DevloCore/tpfinalcid`

Étudiant	Rémi PORQUET
Réalisation	100% (projet individuel)
Repository	<code>https://github.com/DevloCore/tpfinalcid</code>

1 Introduction

Ce rapport détaille la conception, la conteneurisation et la mise en place de l'intégration et du déploiement continus (CI/CD) pour le projet **TaskFlow API**, une API RESTful Node.js dédiée à la gestion de tâches.

L'objectif principal est de démontrer la maîtrise des pipelines d'intégration continue via **Jenkins**, la conteneurisation via **Docker**, et les tests automatisés via **Jest**.

2 Schéma d'Architecture de l'Infrastructure

L'infrastructure repose sur plusieurs composants isolés via Docker Compose :

- **Reverse Proxy Nginx** — écoute sur le port 80 et redirige les requêtes vers l'API.
- **API Node.js** — expose les endpoints REST sur le port interne 5000.
- **Base de données MongoDB** — isolée dans le réseau privé Docker, inaccessible depuis l'hôte.
- **Serveur Jenkins** — orchestre le pipeline CI/CD via le socket Docker de l'hôte.

```
+-----+ +-----+ +-----+ | | | | | Développeur
+----->+ GitHub (Webhook) +----->+ Jenkins | | (Rémi PORQUET) | push | | ping | (Pipeline CI/CD)
| | | | | +-----+ +-----+ +-----+ +-----+ | build &
deploy v +-----+ +-----+ +-----+ +-----+ |
Serveur Hôte (Docker) | | | | +-----+ +-----+ +-----+ +-----+
+-----+ +-----+ | | | Nginx Proxy +--> | TaskFlow API (Node) +--> | MongoDB (Privé) | | | |
(Port : 80) | | (Port interne 5000) | | (Port interne 27017) | | | +-----+
+-----+ +-----+ +-----+ +-----+
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+
```

3 Explication des Choix Techniques

● A. L'Application Node.js et les Tests

- **Express.js & Mongoose** — API développée avec Express pour sa légèreté et Mongoose pour la modélisation stricte des tâches (titre obligatoire, statut via Enum).
- **Couverture de tests à 100%** — Approche de tests unitaires purs avec Jest, en mockant entièrement Mongoose. Ce choix permet d'exécuter les tests très rapidement sans dépendre du téléchargement de binaires MongoDB (incompatibles avec Alpine Linux), garantissant un pipeline robuste et agnostique de l'OS hôte.

● B. Conteneurisation (Docker & Docker Compose)

- **Multi-stage Build** — Le Dockerfile utilise un premier stage *builder* qui installe toutes les dépendances (y compris devDependencies), puis un stage *production* final qui ne copie que le code source et les modules de production. Image basée sur node:18-alpine pour une taille minimale.

- **Isolation Réseau** — MongoDB n'a aucun port exposé vers la machine hôte. Il est uniquement accessible via le réseau taskflow-net, respectant les meilleures pratiques de sécurité.
- **Reverse Proxy** — Nginx écoute sur le port 80 et redirige dynamiquement le trafic vers le conteneur Node.js.

● C. Pipeline CI/CD (Jenkins)

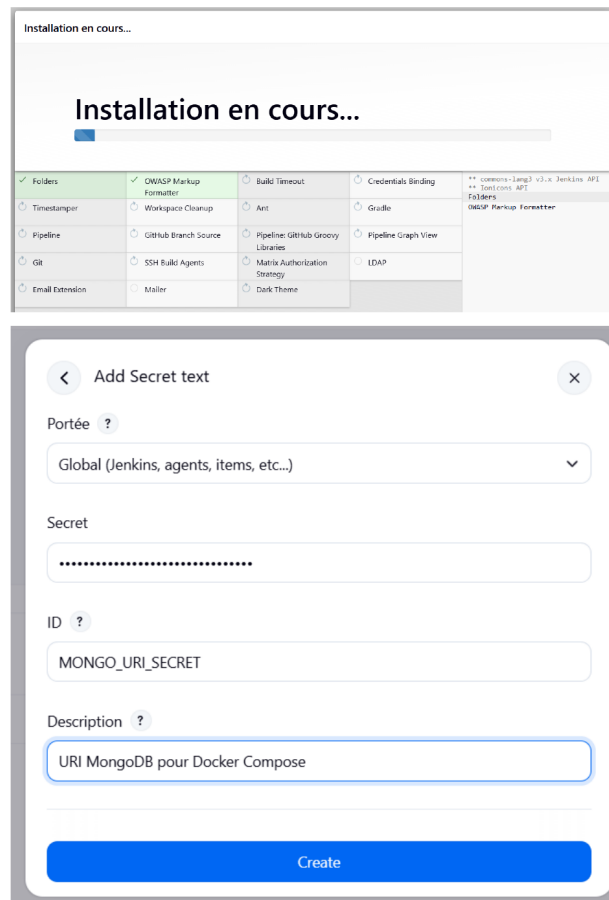
- **Sécurisation des secrets** — L'URI MongoDB n'est jamais écrite en clair dans le code. Elle est injectée dynamiquement via le système de Credentials Jenkins (MONGO_URI_SECRET), évitant toute fuite sur GitHub.
- **Qualité du code** — Le pipeline inclut une étape stricte de Lint (ESLint) avec tolérance zéro. Toute violation de règle (ex. : usage de var) fait immédiatement échouer le build.
- **Agents dynamiques** — Le pipeline utilise des agents Docker éphémères (node:18) pour l'installation, le linting et les tests, garantissant un environnement stérile à chaque build.

4

Configuration Jenkins — Captures d'Écran

● Installation des plugins et configuration du credential

La première étape consiste à installer les plugins Jenkins requis (Pipeline, Git, GitHub, etc.) et à configurer le credential **MONGO_URI_SECRET** de type *Secret text* en portée globale.



Capture 1

Installation plugins Jenkins + création du credential MONGO_URI_SECRET

● Création du job Pipeline « TaskFlow-API »

Un nouveau job de type **Pipeline** est créé dans Jenkins, nommé **TaskFlow-API**. Ce type de job permet d'orchestrer des activités longue durée via un Jenkinsfile versionné dans le dépôt.

Saisissez un nom

TaskFlow-API

Select an item type

-  **Pipeline**
Organise des activités de longue durée qui peuvent s'étendre sur des pipelines (anciennement connues comme workflows) et/ou pas facilement à des tâches de type libre.
-  **Construire un projet free-style**
Job legacy polyvalent qui récupère l'état depuis un outil de gestion suivi d'étapes post-construction telles que l'archivage d'artefacts.
-  **Construire un projet multi-configuration**
Adapté aux projets qui nécessitent un grand nombre de configurations multiples, des binaires spécifiques à une plateforme, etc.
-  **Dossier**
Crée un conteneur qui stocke des objets imbriqués. Utile pour qui n'est qu'un filtre, un dossier crée un espace de nommage commun du même nom tant qu'ils se trouvent dans des dossiers différents.
-  **Organization Folder**
Creates a set of multibranch project subfolders by scanning for...

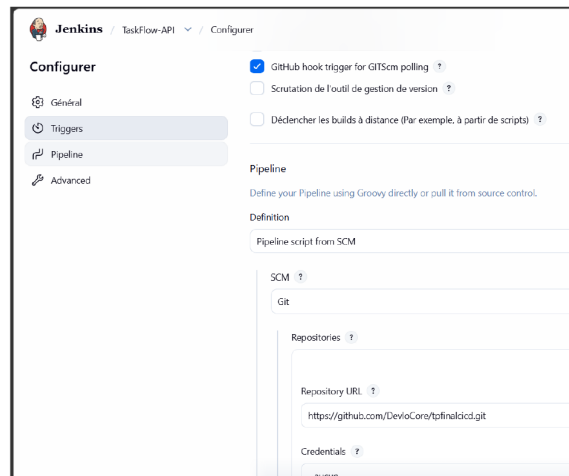
OK

Capture 2

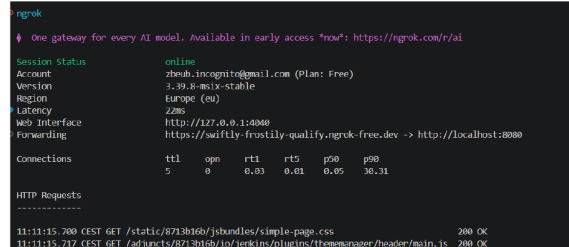
Sélection du type Pipeline lors de la création du job Jenkins

5 Webhook GitHub & Tunnel ngrok

Pour permettre à GitHub de notifier Jenkins lors d'un git push, un tunnel **ngrok** expose le serveur Jenkins local sur une URL HTTPS publique. Le webhook GitHub est ensuite configuré pour envoyer un POST vers cette URL.

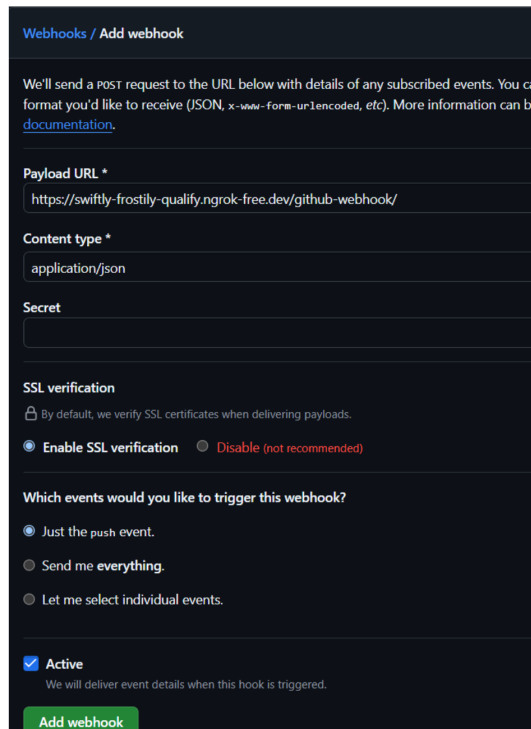


<https://dashboard.ngrok.com/>



Capture 3

Configuration Jenkins SCM + tunnel ngrok actif (forwarding HTTPS → localhost:8080)



Webhooks / Add webhook

We'll send a POST request to the URL below with details of any subscribed events. You can choose the format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in the [documentation](#).

Payload URL *

`https://swiftly-frostily-qualify.ngrok-free.dev/github-webhook/`

Content type *

`application/json`

Secret

SSL verification

By default, we verify SSL certificates when delivering payloads.

☒ Enable SSL verification ☐ Disable (not recommended)

Which events would you like to trigger this webhook?

☒ Just the push event.

☐ Send me everything.

☐ Let me select individual events.

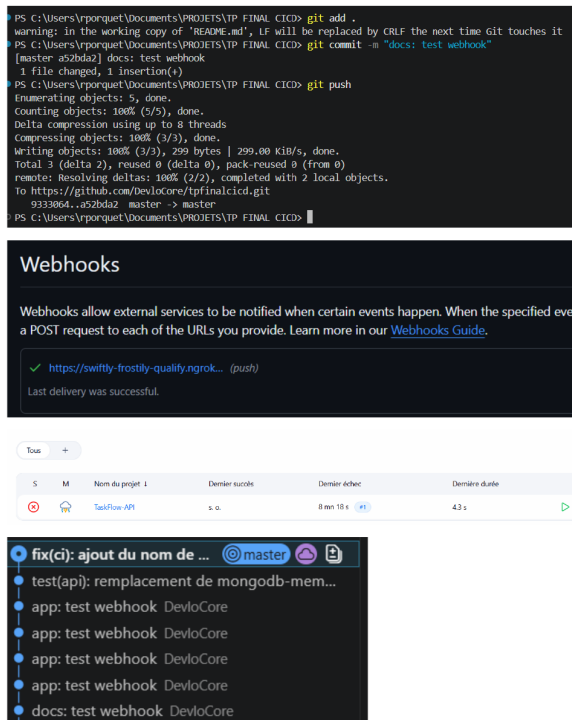
☒ Active

We will deliver event details when this hook is triggered.

Add webhook

Capture 4*Configuration du webhook GitHub — payload URL ngrok, push event, SSL activé***6****Validation du Pipeline CI/CD****● Déclenchement automatique et build vert**

Un git push déclenche automatiquement le webhook, qui notifie Jenkins. Le build #5 s'est terminé en **SUCCESS** en 24 secondes. L'historique des commits confirme l'automatisation complète du cycle.



Capture 5

git push → webhook déclenché → build Jenkins réussi (Last delivery was successful)

● Console Output — Pipeline SUCCESS & build rouge intentionnel

La console Jenkins confirme l'exécution réussie de toutes les étapes : installation, lint, tests (couverture 100%), et déploiement Docker Compose. Le graphe de tendance montre les builds #1 à #4 en rouge (phase de mise au point) et le build #5 en vert (pipeline stabilisé).


```
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline terminé. Le résumé de la couverture des tests est affiché dans la console du stage Test.
[Pipeline] echo
Succès ! L'application est déployée et accessible sur http://localhost/
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Tendance des temps de construction



Capture 6 Console Output SUCCESS + graphe de tendance (builds rouge → build vert final)

7 Tableau de Répartition des Tâches

Le projet ayant été réalisé en solitaire, l'intégralité des tâches a été assumée par un seul étudiant.

Tâche / Composant	Réalisé par	Participation
Initialisation du projet Node.js & Architecture	Rémi PORQUET	100%
Développement de l'API (Routes, Modèles Mongoose)	Rémi PORQUET	100%
Rédaction des Tests (Jest / Supertest) et Mocks	Rémi PORQUET	100%
Configuration ESLint	Rémi PORQUET	100%
Création du Dockerfile (Multi-stage)	Rémi PORQUET	100%
Configuration Docker Compose & Nginx	Rémi PORQUET	100%
Rédaction du Jenkinsfile (Pipeline CI/CD)	Rémi PORQUET	100%
Configuration Jenkins (Credentials, Webhook)	Rémi PORQUET	100%
Rédaction de la documentation (README, Rapport)	Rémi PORQUET	100%

Fin du rapport — Rémi PORQUET · TP Final CI/CD · Juin 2026